MATERIAL DE CONSULTA · TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

Referencial: Estrutura Python/MySQL para Aplicações em Console

Baseado na Documentação Oficial do Python e do MySQL Connector/Python e nos Requisitos de Avaliação

Resumo

Este referencial reúne os elementos essenciais da linguagem Python e da biblioteca MySQL Connector/Python para construir uma aplicação de console com autenticação de usuário, menu de opções e operações de cadastro, consulta, alteração e exclusão de registros (CRUD) em banco de dados MySQL. É um guia de consulta focado na estrutura mínima de cada componente e na criação do banco de dados necessária para a avaliação. Os exemplos foram escritos para Python 3.10 ou superior.

Python · venv · pip · hashlib · MySQL · mysql-connector-python

NESTE REFERENCIAL

01 Preparação do Ambiente
02 Estrutura Base de um Script Python
03 Estruturas de Controle (Menu de Opções)
04 Listas e Funções com Retorno
05 Tratamento de Erros
06 Proteção de Senhas com hashlib
07 Configuração de Conexão com MySQL (db_config.py)
08 Estrutura do Banco de Dados MySQL (SQL)
09 Operações CRUD com Connector/Python
– Referências

Referências: Documentação oficial do Python (https://docs.python.org/pt-br/3/) e MySQL Connector/Python (https://dev.mysql.com/doc/connector-python/en/).

01 Preparação do Ambiente

O ambiente virtual isola as bibliotecas de cada projeto, evitando conflitos de versão entre aplicações. O conector do MySQL é instalado dentro desse ambiente.

1.1 Elementos-chave

`python -m venv .venv` cria o ambiente virtual na pasta .venv.

`activate` ativa o ambiente; o nome dele passa a aparecer no início do terminal.

`pip install` instala um pacote do Python Package Index (PyPI).

`pip freeze > requirements.txt` registra os pacotes instalados, permitindo reinstalá-los com pip install -r requirements.txt.

1.2 Exemplo de Comandos (terminal)

EXEMPLO

```
# Cria o ambiente virtual na pasta do projeto
python -m venv .venv

# Ativa o ambiente (Windows)
.venv\Scripts\activate

# Ativa o ambiente (Linux/macOS)
source .venv/bin/activate

# Instala o conector oficial do MySQL
python -m pip install mysql-connector-python

# Registra as dependências do projeto
python -m pip freeze > requirements.txt

# Desativa o ambiente ao final do trabalho
deactivate
```

02 Estrutura Base de um Script Python

Um script Python é um arquivo .py que pode ser executado como programa ou importado por outros arquivos como módulo. A organização recomendada separa importações, funções e ponto de entrada.

2.1 Elementos-chave

`import / from ... import` traz para o arquivo funções definidas em outros módulos.

`def` define uma função; o corpo é delimitado pela indentação (4 espaços).

`"""docstring"""` texto opcional na primeira linha da função que documenta sua finalidade.

`return` encerra a função e devolve um valor; sem ele, a função retorna None.

`if __name__ == "__main__":` executa o bloco apenas quando o arquivo é rodado diretamente, e não quando é importado.

2.2 Exemplo de Estrutura

EXEMPLO

```
# Arquivo: app.py

# 1. Importações (sempre no início do arquivo)
from db_config import conectar

# 2. Definição de funções
def exibir_titulo(texto):
    """Exibe um título formatado no console."""
    print("=" * 40)
    print(texto)
    print("=" * 40)

# 3. Função principal do programa
def main():
    exibir_titulo("Sistema de Cadastro")

# 4. Ponto de entrada: só executa quando o arquivo é rodado diretamente
if __name__ == "__main__":
    main()
```

03 Estruturas de Controle (Menu de Opções)

As estruturas de controle definem quais instruções são executadas e quantas vezes. Em aplicações de console, são a base do menu de navegação.

3.1 Elementos-chave

`input()` lê o texto digitado pelo usuário (sempre como string).

`if / elif / else` executa apenas o primeiro bloco cuja condição for verdadeira; substitui o switch/case de outras linguagens.

`while` repete o bloco enquanto a condição for verdadeira.

`for ... in` percorre os elementos de uma sequência (lista, string, range()).

`break / continue` break encerra o laço; continue pula para a próxima repetição.

`and / or / not` combinam condições; in e not in testam se um valor está em uma coleção.

3.2 Exemplo de Menu com while e if

EXEMPLO

```
def menu():
    """Exibe o menu e trata a opção escolhida."""
    while True:
        print("\n1 - Cadastrar item")
        print("2 - Listar itens")
        print("0 - Sair")
        opcao = input("Escolha uma opção: ")

        if opcao == "1":
            print("Opção de cadastro selecionada.")
        elif opcao == "2":
            print("Opção de listagem selecionada.")
        elif opcao == "0":
            print("Encerrando o sistema.")
            break          # Sai do laço while
        else:
            print("Opção inválida. Tente novamente.")
```

3.3 Exemplo de Laço for

EXEMPLO

```
itens = ["Mouse", "Teclado", "Monitor"]

# enumerate() devolve a posição e o valor de cada elemento
for posicao, nome in enumerate(itens, start=1):
    print(posicao, nome)
```

RESULTADO ESPERADO NO CONSOLE

```
1 Mouse
2 Teclado
3 Monitor
```

04 Listas e Funções com Retorno

Listas armazenam coleções ordenadas e mutáveis de elementos. Combinadas com funções que retornam valores, permitem organizar e reaproveitar o processamento dos dados.

4.1 Elementos-chave

`lista.append(x)` adiciona um elemento ao final da lista.
`lista.remove(x)` remove a primeira ocorrência de x.
`lista.pop(i)` remove e devolve o elemento da posição i (o último, se omitida).
`lista.sort()` ordena a própria lista.
`len(lista)` devolve a quantidade de elementos.
`{"chave": valor}` dicionário; permite representar um registro com campos nomeados.

4.2 Exemplo de Estrutura

EXEMPLO

```
def filtrar_por_preco(itens, preco_maximo):
    """Retorna uma nova lista com os itens até o preço informado."""
    resultado = []
    for item in itens:
        if item["preco"] <= preco_maximo:
            resultado.append(item)     # Adiciona ao final da lista
    return resultado

catalogo = [
    {"nome": "Mouse", "preco": 25.00},
    {"nome": "Monitor", "preco": 899.90},
]

baratos = filtrar_por_preco(catalogo, 100)
print(len(baratos), "item(ns) encontrado(s)")
```

RESULTADO ESPERADO NO CONSOLE

```
1 item(ns) encontrado(s)
```

05 Tratamento de Erros

A instrução try permite capturar erros em tempo de execução e responder a eles sem encerrar o programa, por exemplo quando o usuário digita um valor inválido ou o banco de dados não está disponível.

5.1 Elementos-chave

`try` bloco com as instruções que podem gerar erro.

`except Tipo as e` trata um tipo específico de erro; e guarda os detalhes da ocorrência.

`else` executado somente quando não ocorre erro no try.

`finally` executado sempre; usado para liberar recursos (ex.: fechar a conexão).

`ValueError` erro de conversão de valor, como float("abc").

5.2 Exemplo de Estrutura

EXEMPLO

```
try:
    preco = float(input("Preço (R$): "))
except ValueError:
    # Executado quando o texto digitado não pode ser convertido
    print("Valor inválido. Use apenas números (ex.: 25.90).")
else:
    # Executado somente se não houve erro no bloco try
    print("Preço registrado:", preco)
finally:
    # Executado sempre, com ou sem erro
    print("Operação finalizada.")
```

06 Proteção de Senhas com hashlib

A senha nunca deve ser armazenada como texto puro. O módulo hashlib gera um resumo (hash) de tamanho fixo; no login, compara-se o hash da senha digitada com o hash armazenado no banco.

6.1 Elementos-chave

`hashlib.sha256()` cria o objeto de resumo com o algoritmo SHA-256.

`.encode("utf-8")` converte o texto em bytes, formato exigido pelo hashlib.

`.hexdigest()` devolve o resumo como texto hexadecimal de 64 caracteres, que pode ser gravado em CHAR(64).

6.2 Exemplo de Estrutura

EXEMPLO

```
import hashlib

def gerar_hash(senha):
    """Retorna o resumo SHA-256 da senha informada."""
    return hashlib.sha256(senha.encode("utf-8")).hexdigest()

print(gerar_hash("senha123"))
```

RESULTADO ESPERADO NO CONSOLE

```
55a5e9e78207b4df8699d60886fa070079463547b095d1a05bc719bb4e6cd251
```

> **Observação:**
>
> para maior segurança em sistemas reais, a senha pode ser combinada com um valor aleatório (salt) por meio da função `hashlib.pbkdf2_hmac()`.

07 Configuração de Conexão com MySQL (db_config.py)

Sempre crie um arquivo separado para as credenciais do banco de dados e importe-o nos demais scripts. A função conectar() concentra a abertura da conexão e o tratamento dos erros mais comuns.

7.1 Elementos-chave

`mysql.connector.connect()` cria a conexão com o servidor MySQL e retorna um objeto de conexão.

`**CONFIG` repassa os itens do dicionário como argumentos nomeados (user, password, host, database).

`mysql.connector.Error` classe base dos erros do conector.

`errorcode` constantes que identificam o tipo de erro, como acesso negado ou banco inexistente.

7.2 Exemplo de Estrutura

EXEMPLO

```
# Arquivo: db_config.py
import mysql.connector
from mysql.connector import errorcode

CONFIG = {
    "user": "seu_usuario",        # Usuário do MySQL (ex.: root)
    "password": "sua_senha",      # Senha do MySQL
    "host": "127.0.0.1",          # Geralmente o computador local
    "database": "nome_do_banco",  # Nome do seu banco de dados
}

def conectar():
    """Retorna uma conexão ativa com o MySQL ou None em caso de falha."""
    try:
        return mysql.connector.connect(**CONFIG)
    except mysql.connector.Error as err:
        if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
            print("Usuário ou senha do banco incorretos.")
        elif err.errno == errorcode.ER_BAD_DB_ERROR:
            print("Banco de dados não encontrado.")
        else:
            print(err)
        return None
```

08 Estrutura do Banco de Dados MySQL (SQL)

A criação do banco de dados e das tabelas é o primeiro passo para o CRUD. Os comandos abaixo podem ser executados em um cliente MySQL (ex.: MySQL Workbench) ou enviados pelo método cursor.execute().

8.1 Criação do Banco de Dados

EXEMPLO

```
-- 1. Cria o banco de dados (se ainda não existir)
CREATE DATABASE IF NOT EXISTS nome_do_banco DEFAULT CHARACTER SET utf8mb4;

-- 2. Seleciona o banco de dados para uso
USE nome_do_banco;
```

8.2 Criação de Tabelas, Chave Primária e Chave Estrangeira

A chave primária (PRIMARY KEY) identifica unicamente cada registro e o AUTO_INCREMENT gera os identificadores automaticamente. A chave estrangeira (FOREIGN KEY) cria o relacionamento entre tabelas, garantindo a integridade referencial.

EXEMPLO

```
-- Tabela 1: Usuarios (para o login)
CREATE TABLE IF NOT EXISTS usuarios (
    id INT NOT NULL AUTO_INCREMENT,
    usuario VARCHAR(50) NOT NULL UNIQUE,
    senha CHAR(64) NOT NULL,           -- Hash SHA-256 (64 caracteres)
    nome_completo VARCHAR(100),
    PRIMARY KEY (id)
);

-- Tabela 2: Categorias (necessária para o relacionamento)
CREATE TABLE IF NOT EXISTS categorias (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50) NOT NULL UNIQUE,
    PRIMARY KEY (id)
);

-- Tabela 3: Itens (alvo do CRUD)
CREATE TABLE IF NOT EXISTS itens (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    descricao TEXT,
    preco DECIMAL(10, 2) NOT NULL,
    categoria_id INT,                  -- Campo da chave estrangeira
    PRIMARY KEY (id),
    CONSTRAINT fk_categoria FOREIGN KEY (categoria_id)
        REFERENCES categorias (id)
        ON DELETE RESTRICT             -- Impede excluir categoria em uso
        ON UPDATE CASCADE              -- Propaga alteração do id
);

-- Usuário de teste (hash SHA-256 de "senha123", ver Seção 6)
INSERT INTO usuarios (usuario, senha, nome_completo)
VALUES ('admin',
        '55a5e9e78207b4df8699d60886fa070079463547b095d1a05bc719bb4e6cd251',
        'Administrador');
```

09 Operações CRUD com Connector/Python

Todas as instruções SQL são enviadas por meio de um cursor. O Connector/Python não confirma alterações automaticamente: após INSERT, UPDATE ou DELETE é necessário chamar commit(), ou rollback() para desfazê-las.

9.1 Elementos-chave

`cnx.cursor()` cria o cursor que executa os comandos SQL.

`cursor.execute(sql, valores)` executa o comando; os valores são passados em tupla e substituem os marcadores %s, protegendo contra injeção de SQL.

`cursor.fetchone() / fetchall()` devolvem uma linha ou todas as linhas do resultado.

`cursor.lastrowid` identificador gerado pelo último INSERT.

`cursor.rowcount` quantidade de linhas afetadas pelo último comando.

`cnx.commit() / cnx.rollback()` confirmam ou desfazem as alterações.

`close()` encerra o cursor e a conexão ao final de cada operação.

9.2 Validação de Login

EXEMPLO

```
# Arquivo: autenticacao.py
import hashlib
from db_config import conectar

def autenticar(usuario, senha):
    """Retorna True quando usuário e senha conferem com o cadastro."""
    cnx = conectar()
    if cnx is None:
        return False

    cursor = cnx.cursor()
    sql = "SELECT senha FROM usuarios WHERE usuario = %s"
    cursor.execute(sql, (usuario,))     # Parâmetro sempre em tupla
    registro = cursor.fetchone()        # Uma linha ou None
    cursor.close()
    cnx.close()

    if registro is None:
        return False
    senha_hash = hashlib.sha256(senha.encode("utf-8")).hexdigest()
    return senha_hash == registro[0]
```

9.3 Cadastro (Create) e Listagem (Read)

EXEMPLO

```
# Arquivo: itens.py
import mysql.connector
from db_config import conectar

def cadastrar_item(nome, descricao, preco):
    """CREATE: insere um novo item na tabela itens."""
    cnx = conectar()
    cursor = cnx.cursor()
    sql = ("INSERT INTO itens (nome, descricao, preco) "
           "VALUES (%s, %s, %s)")
    try:
        cursor.execute(sql, (nome, descricao, preco))
        cnx.commit()                    # Confirma a gravação
        print("Item cadastrado. ID:", cursor.lastrowid)
    except mysql.connector.Error as err:
        cnx.rollback()                  # Desfaz em caso de erro
        print("Falha ao cadastrar:", err)
    finally:
        cursor.close()
        cnx.close()

def listar_itens():
    """READ: exibe os itens cadastrados em ordem alfabética."""
    cnx = conectar()
    cursor = cnx.cursor()
    cursor.execute("SELECT id, nome, preco FROM itens ORDER BY nome")
    for (id_item, nome, preco) in cursor:
        print(f"{id_item:<4} {nome:<20} R$ {preco:.2f}")
    cursor.close()
    cnx.close()
```

RESULTADO ESPERADO NO CONSOLE

```
1    Mouse Óptico         R$ 25.00
2    Teclado              R$ 89.90
```

9.4 Alteração e Exclusão (Dicas)

Update: recebe o identificador do registro, executa `UPDATE itens SET nome = %s, preco = %s WHERE id = %s` com os valores em tupla e confirma com `cnx.commit()`.

Delete: recebe o identificador e executa `DELETE FROM itens WHERE id = %s`, também seguido de `cnx.commit()`.

Verificação: após UPDATE ou DELETE, `cursor.rowcount` igual a 0 indica que nenhum registro com o identificador informado foi encontrado.

Referências

ORACLE CORPORATION. MySQL Connector/Python Developer Guide. Disponível em: https://dev.mysql.com/doc/connector-python/en/. Acesso em: 18 out. 2026.

PYTHON SOFTWARE FOUNDATION. hashlib: resumos de mensagens e hashes seguros. Disponível em: https://docs.python.org/pt-br/3/library/hashlib.html. Acesso em: 18 out. 2026.

PYTHON SOFTWARE FOUNDATION. Tutorial do Python. Disponível em: https://docs.python.org/pt-br/3/tutorial/index.html. Acesso em: 18 out. 2026.